ЛИДЕРЫ ЦИФРОВОЙ ТРАНСФОРМАЦИИ 2026 · СПЕЦТРЕК ФСП

ФСП Таланты

Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника ФСП. Сопроводительная документация.

Стенд	https://fsp-talents.vercel.app (из части российских сетей может потребоваться VPN — согласовано с организаторами)
Репозиторий	https://github.com/m3ksksssssss/FSP-Talents
API (Swagger UI)	https://fsp-talents.vercel.app/api/docs
Демо-аккаунты	candidate@demo.ru, strong@demo.ru, employer@demo.ru, admin@demo.ru · пароль Demo12345

Где найти требуемое по п. 3.4 ТЗ

Требование ТЗ	Раздел
Функциональная и компонентная архитектура	2. Архитектура
Механика тестирования и устойчивость к распространению заданий	3. Механика тестирования; 5. Регулярные задачи
Механика подбора и логика категоризации	4. Механика подбора и категоризация
Процедура валидации и её результаты	6. Валидация и результаты
Схема интеграции с реестром ФСП	7. Интеграция с ФСП
Описание API	8. API (полная спецификация — OpenAPI, /api/docs)
Библиотеки и компоненты с версиями	11. Библиотеки и версии
Сборка, развёртывание, локальный запуск	1. Обзор, запуск и развёртывание
Дополнительно: защита от фиктивных вакансий, ATS	9. Безопасность; 10. Интеграция с ATS

1. Обзор, запуск и развёртывание

Прототип платформы для кейса Федерации спортивного программирования (модель LHH):

1. Кандидат заполняет профиль или загружает PDF-резюме, **сам выбирает грейд** и проходит **адаптивный тест** на него.
2. По результату получает **категорию** «специализация · грейд». Только с ней его видят работодатели.
3. Работодатель описывает потребность (форма или текст вакансии), получает **подборку с доводами «за» и «против»**, уточняет фильтрами и **сам приглашает** кандидата с обязательной вилкой зарплаты.
4. Кандидат принимает или отклоняет. **Контакты открываются только после принятия.**
5. Достижения в соревнованиях ФСП подтверждаются (реестр, сертификат, репозиторий) и поднимают кандидата внутри его категории.
6. Раз в 14 дней кандидат получает короткую задачу от работодателя своей категории: свежий результат виден в подборке.
7. После согласия кандидата стороны общаются в чате и проводят видеоинтервью с общим редактором кода (LiveKit). Тест и интервью — только с компьютера.

Стенд: https://fsp-talents.vercel.app (Vercel + Neon, демо-аккаунты ниже). Из некоторых российских сетей домен `*.vercel.app` может открываться только через VPN — это согласовано с организаторами; без VPN решение поднимается локально одной командой (ниже).

Быстрый старт (Docker)

```
docker compose up --build -d                    # app + Postgres 16 + Mailpit (миграции применяются при старте)
docker compose --profile seed run --rm seed     # синтетические данные + демо-аккаунты (≈ 10 с)
```

Что	Адрес
Приложение	http://localhost:3000
Почта (Mailpit)	http://localhost:8025
API (Swagger UI)	http://localhost:3000/api/docs
Keycloak «ФСП ID» (по желанию)	`AUTH_KEYCLOAK_ENABLED=true docker compose --profile keycloak up -d` → http://localhost:8080

Демо-аккаунты

Пароль у всех: `Demo12345`

Email	Роль	Что показывает
`candidate@demo.ru`	кандидат без категории	путь «резюме → тест → категория»
`strong@demo.ru`	кандидат с категорией Backend · Middle и достижениями ФСП	кабинет, входящие приглашения, новая задача от работодателя

Email	Роль	Что показывает
`employer@demo.ru`	работодатель «Кванта Софт»	потребности, подборка, приглашения, вакансии, регулярные задачи
`admin@demo.ru`	модератор	попытки по индексу доверия, ловушки, банк заданий, похожие решения

В синтетике ещё 320 кандидатов (`cand001…@seed.test`) и 32 работодателя (`emp01…@seed.test`) с тем же паролем.

Сценарий показа на 6–7 минут — `docs/demo-script.md`.

Запуск без Docker

Нужны Node.js 22 и PostgreSQL 16.

```
cp .env.example .env.local      # заполните AUTH_SECRET; LLM_API_KEY — по желанию
npm ci
npm run db:migrate
npm run db:seed                 # --reset для полной перегенерации
npm run dev                     # http://localhost:3000
```

Без `LLM_API_KEY` все LLM-функции (разбор вакансии и резюме, перефразирование доводов) работают на детерминированных правилах.

Переменные окружения

Все переменные с комментариями — в `.env.example`. Главные:

Переменная	Назначение
`DATABASE_URL`	PostgreSQL (Neon в проде)
`AUTH_SECRET`, `APP_URL`, `AUTH_TRUST_HOST`	Auth.js, ссылки в письмах
`DEMO_MODE`	Ссылка подтверждения email на экране, подсказка демо-аккаунтов
`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`, `LLM_MODEL_STRONG`, `LLM_EMBEDDING_MODEL`, `LLM_TIMEOUT_MS`	RouterAI (OpenAI-совместимый API)
`SMTP_HOST` / `RESEND_API_KEY`, `MAIL_FROM`	Почта: Mailpit локально, Resend на Vercel, иначе лог
`BLOB_READ_WRITE_TOKEN`, `STORAGE_DIR`	Файлы: Vercel Blob или локальная папка
`AUTH_KEYCLOAK_*`	Вход через «ФСП ID» (по умолчанию выключен)
`LIVEKIT_URL`, `LIVEKIT_API_KEY`, `LIVEKIT_API_SECRET`	Видеоинтервью (LiveKit Cloud или свой сервер)
`UPSTASH_REDIS_REST_*`	Распределённый rate limit (по желанию)
`GITHUB_TOKEN`	Лимит GitHub API для проверки репозиториев (по желанию)

Тесты, валидация, документация

```
npm run typecheck && npm run lint
npm test              # Vitest: 480+ проверок (банк заданий, IRT/CAT, скоринг, сервисы на
реальной БД)
                      # нужна БД TEST_DATABASE_URL (по умолчанию
postgres://fsp:fsp@localhost:5432/fsp_test)
npm run e2e           # Playwright против запущенного приложения (E2E_BASE_URL,
E2E_DATABASE_URL)
npm run validate      # метрики подбора и теста → docs/validation.md + графики (≈ 40 с, без БД)
npm run calibrate     # калибровка IRT-параметров шаблонов → task_templates
npm run openapi       # docs/openapi.json
npm run docs:libs     # docs/libraries.md
```

CI (GitHub Actions) на каждый push:

1. типы, линт, Vitest, сборка;
2. сквозной сценарий Playwright против `docker compose` с наполненной базой: регистрация → тест → категория → поиск → приглашение → принятие → контакты.

Деплой на Vercel

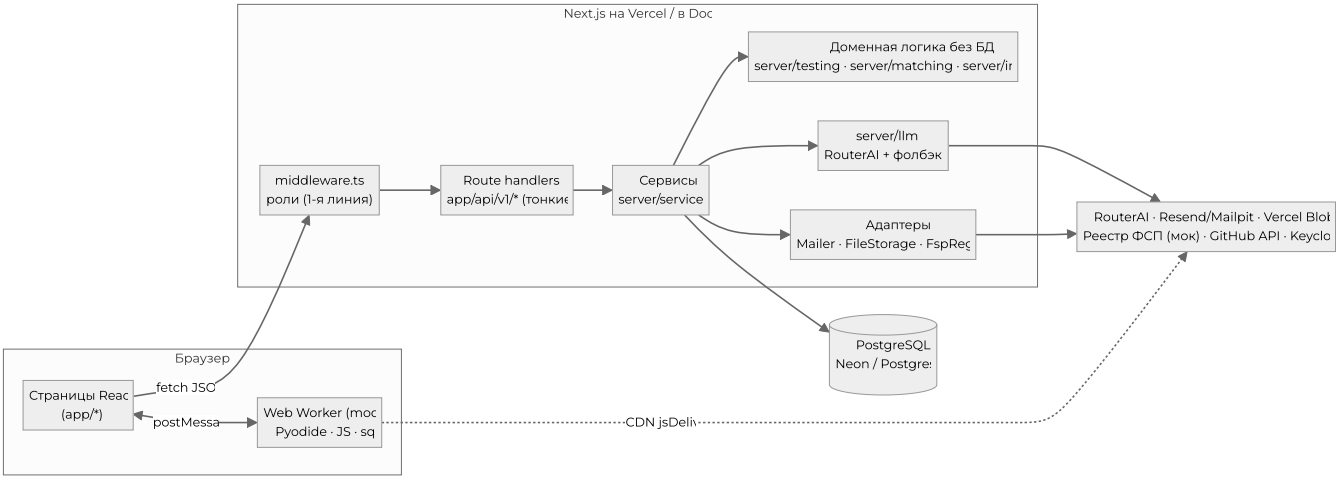
1. Импортируйте репозиторий в Vercel (фреймворк определится сам).
2. Storage → интеграция **Neon** (выставит `DATABASE_URL`) и **Blob** (`BLOB_READ_WRITE_TOKEN`).
3. Задайте `AUTH_SECRET`, `APP_URL`, `DEMO_MODE=true`, `LLM_API_KEY` и при желании `RESEND_API_KEY`.
4. Сборка использует `vercel-build`: миграции, затем **полный сброс базы и заново сид** (`seed.ts --reset`), потом `next build`. Каждый деплой начинается с чистых демо-данных — всё, что пользователи создали на стенде, стирается.

Стек

Next.js 15 (App Router) · TypeScript strict · PostgreSQL + Drizzle · Auth.js v5 (argon2id, Keycloak OIDC) · Zod → OpenAPI 3.1 · RouterAI (пакет `openai`) · Pyodide / sql.js в Web Worker · CodeMirror 6 · Tailwind CSS v4 · Vitest · Playwright · Docker Compose.

2. Архитектура

Монолит на Next.js 15 (App Router) с явным разделением слоёв. Один деплой на Vercel, одна база PostgreSQL.

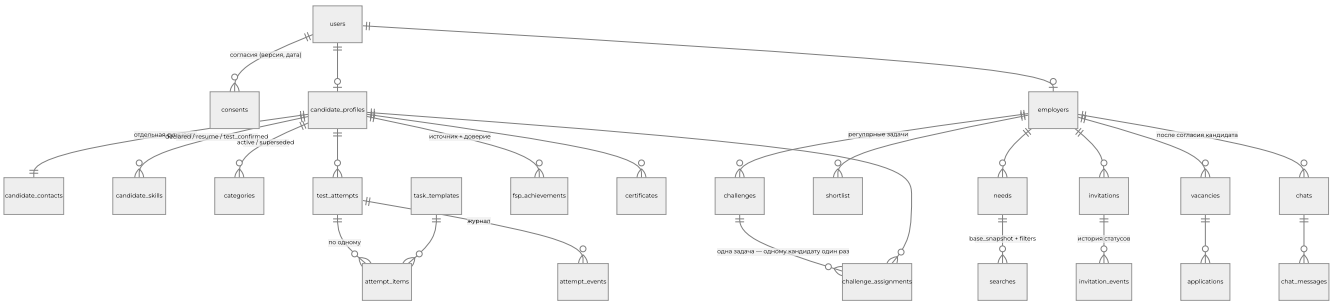


Слои

Слой	Где	Что делает	Чего не делает
UI	`app/`, `components/`	Серверные компоненты читают данные через сервисы, клиентские — через API	Не хранит эталонов, не решает доступ
Route handlers	`app/api/v1/**/route.ts`	Аутентификация (`server/http/guard.ts`), Zod-валидация (`parseJson` / `parseQuery`), маппинг ошибок в HTTP (`server/http/route.ts`)	Бизнес-логики нет: 5–15 строк на эндпоинт
Сервисы	`server/services/*`	Бизнес-правила, транзакции, проверка владельца, аудит, уведомления	Не знают про HTTP
Домен без БД	`server/testing`, `server/matching`, `server/invitations/fsm.ts`, `server/evidence`	IRT/CAT, генераторы заданий, проверка ответов, скоринг, конечный автомат, ловушки, калибровка	Чистые функции — тестируются и симулируются без базы
Данные	`server/db/schema.ts`, `drizzle/`	Схема Drizzle, миграции	—
Адаптеры	`server/mail`, `server/storage`, `server/fsp`, `server/llm`	Интерфейс + 2 реализации (локально / Vercel / мок)	—

Права проверяются дважды: `middleware.ts` отсекает чужие разделы по роли из JWT, а сервисы перечитывают роль и статус пользователя из БД на каждом запросе (`requireActor`) и проверяют владельца ресурса. Чужие ресурсы возвращают 404 — так нельзя узнать, существует ли объект.

Модель данных



Ключевые решения:

- **Контакты — отдельная таблица `candidate_contacts`.** Сериализатор работодателя (`toCard`, `candidateForEmployer`) не обращается к ней без проверки `hasContactAccess`. Каждое чтение и раскрытие контактов попадает в `audit_log`.
- **Подборка — неизменяемый снимок** (`searches.base_snapshot`): список id и скоров на момент поиска. Фильтры хранятся отдельно и применяются поверх снимка.
- **Генераторы заданий — код**, а метаданные шаблонов (IRT a/b, экспозиция, счётчики для детектора слива, статус) — строки `task_templates`. Синхронизация идёт через `syncTemplates()`, откалиброванные параметры не перетираются.
- **Эталоны не хранятся.** Вариант задания — детерминированная функция `(шаблон, seed задания)`, сервер пересчитывает эталон при проверке.
- **Периодические процессы без cron.** Протухание приглашений и попыток, «свежесть» профиля и выдача регулярных задач считаются лениво при чтении (ограничение бесплатного Vercel).

Ограничения Vercel и как они учтены

Ограничение	Решение
Нет фоновых воркеров	Всё короткими запросами; ленивое протухание; калибровка — CLI-скрипт
Таймаут функций	LLM: таймаут 20 с, `maxDuration = 30`, один вызов на запрос, кеш по хешу входа, детерминированный фолбэк
Нет файловой системы	`FileStorage`: Vercel Blob в проде, `./storage` локально; лимит загрузки 4 МБ
COOP/COEP для прерывания Pyodide	Заголовки только на `/test/*` и `/runner/*`; на страницу теста — полная навигация; module-воркер
Пул соединений	`postgres` с `prepare: false` (pgbouncer Neon), `max: 1` в serverless
In-memory rate limit — на инстанс	Upstash при наличии ключа; суточный лимит приглашений — в БД

3. Механика тестирования

Коротко

1. Кандидат **сам выбирает грейд** и проходит адаптивный тест на него: 10–18 заданий, не больше 60 минут, задания выдаются **по одному сервером**.
2. Задания — **параметризованные шаблоны** с откалиброванной сложностью (IRT 2PL), а не фиксированный список вопросов. Каждый вариант уникален для попытки.
3. Код кандидата запускается **у него в браузере**: Pyodide, JS, sql.js. Но итоговая проверка идёт по **скрытым входам, сгенерированным в момент отправки**. Эталон считает только сервер.
4. По итогу присваивается **категория** «специализация · грейд». Без неё кандидата нет в подборках.
5. Попытку сопровождают **журнал и ловушки для ИИ**, из них складывается **индекс доверия**. Автоматических банов нет.

Банк шаблонов

Специализация	Шаблонов	code	sql	calc	quiz
Backend	27	11 (Python/JS)	6	5	5
Frontend	26	11 (JS)	—	8	7
Data / ML	26	10 (Python)	5	6	5
QA	25	8 (Python/JS)	4	7	6

Шаблон — это TypeScript-модуль (`server/testing/bank/<spec>/*.ts`). По `seed` он выдаёт:

- параметры (числа, константы условия, наборы данных);
- текст условия из блоков (абзацы, код, таблицы);
- открытые примеры с ответами;
- генератор **скрытых входов** (отдельный поток RNG `seed/hidden/nonce`);
- серверный решатель `solve` и **эталонные решения** на Python/JS.

Юнит-тесты (`tests/unit/bank.test.ts`, 411 проверок) для каждого шаблона и пяти seed проверяют:

- детерминизм и разнообразие вариантов;
- что эталонные решения на JS (в Node) и на Python (`python3`) дают ровно то же, что серверный `solve`, на открытых и скрытых входах;
- что эталонный ответ принимается, а неверный — нет;
- что ловушки срабатывают.

Ограничения банка: концептуальных вопросов (quiz) в банке меньше 30%, они выдаются не чаще 30% на любом префиксе теста и весят меньше (вес правдоподобия 0.6). Сложность `b` распределена от −2.2 до +2.3.

Адаптивный тест (CAT)

- **Модель**: 2PL, P(верно | θ) = 1 / (1 + e^(−a(θ − b))).

- **Оценка θ**: EAP на сетке [−4, 4], априорное N(θ_старт/2, 1.25²). Выбор грейда — информативный, но не доминирующий. Частичный балл за код (доля пройденных скрытых тестов) входит в правдоподобие как дробный ответ.
- **Выбор задания**: максимум информации Фишера в текущей θ с поправками:
  - баланс типов (доля типа в банке), запрет трёх одинаковых типов подряд;
  - quiz ≤ 30% на каждом префиксе, вес quiz понижен;
  - **контроль экспозиции**: штраф для часто выдаваемых шаблонов и случайный выбор из top-5 (randomesque).
- **Остановка**: не раньше 10 заданий; затем при SE < 0.42 или когда решение очевидно (|θ − cut| ≥ 2·SE); максимум 18 заданий; серверный дедлайн 60 минут.
- **Балл /30**: ожидаемое число верных ответов на виртуальном эталонном бланке грейда из 30 заданий (true-score equating). Порог грейда θ = cut даёт 18/30, «уверенно» — от 25/30. Так результаты разных адаптивных попыток сопоставимы.
- **Пороги** (`server/config.ts`): cut_junior = −0.9, cut_middle = 0.2, cut_senior = 1.3.

Правила пересдачи (`server/testing/eligibility.ts`, покрыты тестами)

Ситуация	Правило
Первая попытка в специализации	Любой грейд
Тот же грейд	Не чаще раза в 14 дней
Грейд ниже уже пробованного	Сразу (понижение не принудительное)
Грейд выше всех пробованных	Только при результате ≥ 25/30 на грейде ниже
Грейд выше активной категории	Первые 7 дней после первой попытки — сразу (кандидат ищет свой уровень); потом — не раньше чем через 90 дней после присвоения категории

Почему так. ТЗ требует ограничить смену грейда по времени, а на Q&A уточнили: нельзя «сегодня Junior, завтра Senior». Неделя калибровки нужна, чтобы кандидат, недооценивший себя на старте, не ждал три месяца. 90 дней — срок, за который реально вырасти на грейд; чаще менять категорию означало бы проверять не рост, а удачу. Понижение и пересдача ниже не ограничены: попытка не «сгорает».

Провал не снимает существующую категорию. Новая категория заменяет старую той же специализации, только если её грейд не ниже.

Экспресс-проверка для жюри (только `DEMO_MODE`)

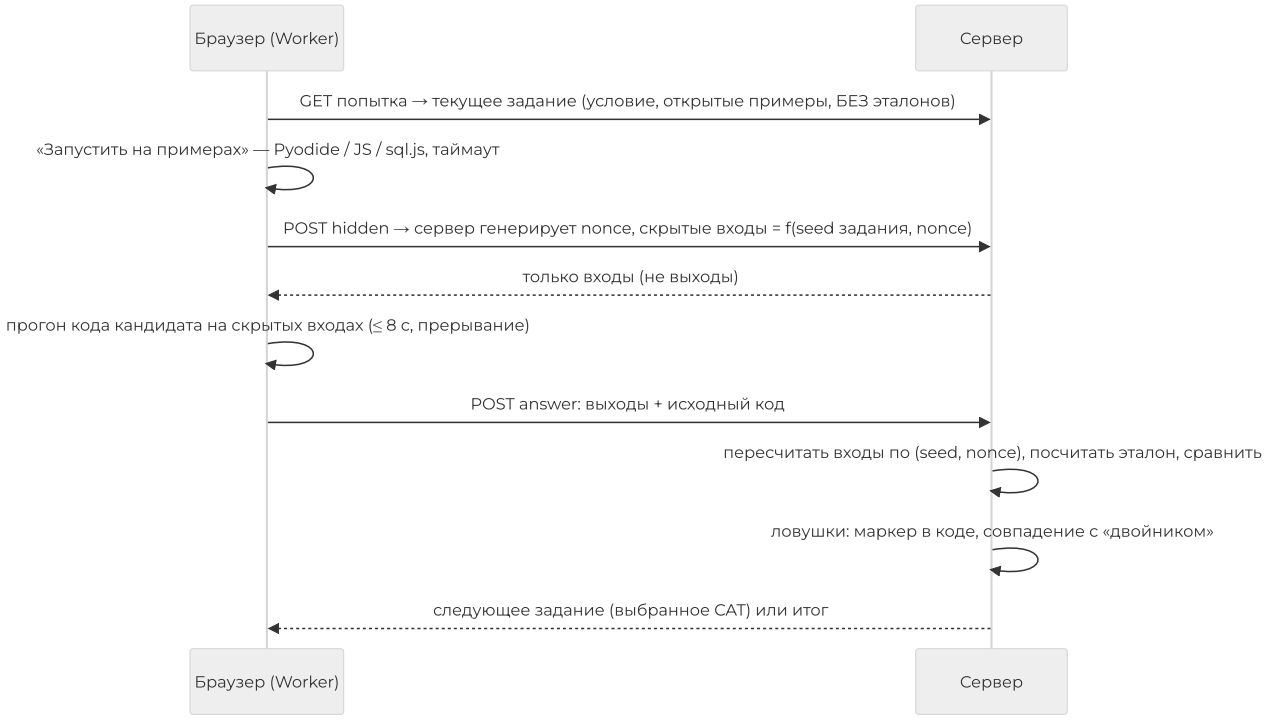
На Q&A жюри попросило возможность самим пройти тест за 3–5 минут. Экспресс-проверка — это тот же движок CAT, та же шкала /30 и те же правила присвоения категории, но:

	Полный тест	Экспресс-проверка
Заданий	10–18, адаптивная остановка	ровно 6
Типы	code, sql, calc, quiz	только calc и quiz (без запуска кода)
Время	60 минут	15 минут
Категория	обычная	с пометкой «экспресс-проверка» в выдаче

	Полный тест	Экспресс-проверка
Счётчики калибровки, экспозиция, детектор слива	обновляются	не трогаются, чтобы демо-прохождения не искажали параметры заданий
Пересдачи и кулдауны	по правилам выше	не учитываются и не расходуются

Вне демо-режима API отвечает 403. Точность на 6 заданиях заметно ниже, поэтому на странице результата это сказано прямо.

Проверка кода: «подделать можно, только решив задачу»



- Скрытые входы появляются только после нажатия «Отправить», живут 120 секунд, на каждую отправку генерируется новый nonce.
- Для SQL сервер сам выполняет эталонный запрос на сгенерированных данных через тот же sql.js.
- Код сохраняется для антиплагиата и перепроверки модератором.
- Python прерывается через interrupt-буфер (SharedArrayBuffer при COOP/COEP) без перезагрузки Pyodide. JS и SQL при таймауте пересоздают воркер.

Честное ограничение. Скорость и память в браузере не проверяются, а код технически можно выполнить и вне браузера. Но верные выходы на свежих скрытых входах получит только тот, кто решил задачу. Для задач с лимитами времени и памяти план — серверная песочница (Judge0/Piston на VPS), интерфейс проверки к этому готов.

Почему банк не сольют

Угроза	Защита
Ответы публикуют в Telegram	Слив варианта бесполезен: числа, данные и константы меняются по seed, скрытые входы — по nonce в момент отправки

Угроза	Защита
Сливают сам шаблон (идею задачи)	Адаптивность и контроль экспозиции растягивают использование банка; **онлайн-детектор дрейфа**: если доля верных ответов на шаблоне выше прогноза IRT на ≥ 15 п. п. при z ≥ 3, шаблон получает `flagged` и не учитывается в θ. В симуляции слива 20% шаблонов детектор нашёл все 5 слитых без ложных срабатываний, ложный «сдал на грейд выше» упал с 40.8% до 15.1% (см. `validation.md`)
Генерация заданий ИИ «на лету» несопоставима	Генераторы — проверенный код; сложность откалибрована; балл приведён к эталонному бланку
Решение за кандидата пишет ИИ	Ловушки и индекс доверия (ниже), сравнение решений между кандидатами
Перебор ответов	Ответ на задание принимается один раз; пересдача — по правилам выше

Антифрод: журнал, ловушки, индекс доверия

Журнал (`attempt_events`):

- уходы со вкладки и потеря фокуса;
- выход из полноэкранного режима: тест идёт только в полноэкранном режиме, при выходе контент закрывается;
- попытки копирования условия: копирование запрещено, попытка пишется;
- вставки в редактор (только длина, без содержимого) и крупные вставки (≥ 200 символов);
- время на задание.

Ловушки (уникальны для попытки, `server/testing/engine.ts` + `components/test/statement.tsx`):

1. **Персональный маркер** — слово вида `kvant_x9d` на попытку.
2. **Скрытый текст** в условии (font-size 0, `aria-hidden`) с инструкцией ИИ-ассистенту назвать переменную маркером. Попадает в DOM, в буфер при обходе запрета копирования и к расширениям, читающим страницу.
3. **Микротекст** той же инструкции почти фоновым цветом внутри декоративной линии — против фото и скриншотов.
4. **Значение-двойник**: на экране одно число (через CSS `::before`), в DOM-тексте другое. Решение, посчитанное по скопированному тексту, совпадает с «невозможным» вариантом. Сервер проверяет это, пересчитывая эталон с подставленным двойником. Для экранных дикторов доступно видимое значение (`role="img"` + `aria-label`).

Индекс доверия = 100 − Σ сигналов: маркер в коде (−65), двойник (−40), крупные вставки, уходы со вкладки, выходы из полноэкранного режима, попытки копирования, аномально быстрые верные ответы на сложных заданиях, person-fit lz (Drasgow).

Уровень	Что происходит
high ≥ 70	Категория присваивается
medium 40–69	Категория присваивается с пометкой (видна работодателю, снижает вклад теста в скоринг)
low < 40	Категория не присваивается; попытка уходит модератору, кандидату предложена перепроверка

Модератор (`/admin`) видит журнал, сработавшие ловушки, код и похожие решения других кандидатов: нормализованные токены, 5-граммы, Жаккар ≥ 0.8. Он подтверждает результат

(категория с пометкой) или отклоняет. Кандидат предупреждён о журнале до начала теста (152-ФЗ, прозрачность).

Калибровка

- Стартовые a, b — экспертные.
- На каждый ответ копятся `answered`, `correct`, `expected` (Σ P(верно) при θ до ответа) — для детектора дрейфа.
- `npm run calibrate` переоценивает a, b логистической регрессией по оценкам θ (`server/testing/calibration.ts`). Берёт реальные ответы, если их ≥ 30 на шаблон, иначе синтетику. Откалиброванные параметры не перетираются синхронизацией шаблонов.
- В валидации калибровка снижает RMSE параметра b с 0.47 до 0.19 на симулированных данных.

Что показала валидация (см. `validation.md`)

- Тест-ретест на разных вариантах: корреляция θ 0.81, совпадение решения «сдал / не сдал» ~79%.
- Итоговая категория совпала с истинным уровнем у ~76% кандидатов, ошибки — на одну ступень у границ грейдов. Категорию выше истинной получили ~7%.
- Точность ограничена информативностью банка (26 шаблонов на специализацию): CAT почти всегда использует максимум заданий. Рост банка и калибровка на реальных ответах — главный рычаг повышения точности.

4. Механика подбора и категоризация

Код: `server/matching/score.ts` (чистые функции), `server/services/matching.ts` (выборка, снимок, фильтры). Тесты: `tests/unit/score.test.ts`, `tests/integration/matching.test.ts`.

1. Ввод потребности

Работодатель заполняет форму или вставляет текст вакансии. Текст разбирается так:

- **LLM (RouterAI)** со структурированным выводом по Zod-схеме (`response_format: json_schema`). Если схема не поддерживается — JSON-режим, валидация и одна попытка исправления. Кеш по хешу входа, таймаут 20 с.
- **Фолбэк без LLM**: словарь стеков (~65 навыков с алиасами RU/EN), регулярные выражения для грейда (в том числе по фразе «опыт от N лет»), зарплаты («от 150 000 до 250 000 ₽», «180–260 тыс.», «200k-300k»), формата и города.

Результат показывается в форме для подтверждения. Без полной вилки сохранить потребность нельзя (422).

2. Жёсткий фильтр (правило 5)

В пул попадают только кандидаты, у которых:

- есть **активная категория из теста** нужной специализации и грейда (соседний грейд — отдельным блоком «Близкие кандидаты»);
- `visibility = visible`, аккаунт не удалён;
- действует согласие на обработку ПДн.

Самоописание и резюме не добавляют кандидата в пул. Они влияют только на ранжирование, и такие навыки всегда помечены «заявлено».

Неподтверждённый грейд. Кандидат, прошедший тест на грейд потребности, но не набравший порог, не исчезает: он идёт отдельным блоком «Грейд не подтверждён» после всех подтверждённых, с бейджем и фактическим баллом, а первый довод «против» — «Грейд Middle не подтверждён: 16.0/30 при пороге 18». Так советовали на Q&A: иначе кандидат выпадает из оборота. Условия: последняя завершённая попытка в специализации провалена, ответов ≥ 5, индекс доверия не low (такие попытки сначала смотрит модератор), активной категории в специализации нет. Блок отключается фильтром «Показывать неподтверждённый грейд». Это тоже результат теста, а не резюме, поэтому правило 5 не нарушается.

3. Скоринг (детерминированный, объяснимый)

match = round(100 · Σ $w_i \cdot c_i$), где все $c_i \in [0, 1]$, а веса заданы в `MATCH_WEIGHTS` (`server/config.ts`):

Компонента	Вес	Как считается
`test`	0.40	(балл/30 − 12) / 18, обрезка в [0, 1]; при среднем доверии ×0.9
`stack`	0.22	Среднее по навыкам потребности: подтверждён тестом 1.0, косвенно подтверждён тестом (SQL → PostgreSQL) 0.8, из резюме 0.6, заявлен 0.5, нет данных 0
`fsp`	0.14	raw / (raw + 1.5), см. ниже

Компонента	Вес	Как считается
`salary`	0.10	Ожидания ≤ верха вилки → 1; выше — штраф 1 − 3·превышение (не исключение); ожиданий нет → 0.75
`formatCity`	0.07	Совпадение формата 1; гибрид ↔ другое 0.6; офис ↔ удалёнка 0.25; другой город при офисе/гибриде ×0.35/0.5
`freshness`	0.07	0.6·свежесть теста (1 до 90 дней, затем экспоненциальный спад) + 0.4·активность профиля

Бонус ФСП с убывающей отдачей

Вес достижения = место × доверие × этап × свежесть, где:

- место: 1-е = 1.0, 2-е = 0.75, 3-е = 0.6, 4–10 = 0.3, 11–50 = 0.12, участие без места = 0.02;
- доверие: реестр ФСП 1.0 > организатор 0.8 > репозиторий 0.6 > заявлено 0.2;
- этап: финал 1.0, окружной 0.8, региональный 0.6;
- свежесть: полураспад 3 года.

raw = Σ(top-5 весов) + 0.5 · ln(1 + Σ остальных весов).

Одна победа весит больше сотни участий без мест — это проверяет тест: 1.0 против 5·0.02 + 0.5·ln(1 + 95·0.02) ≈ 0.63. Кандидат без истории в ФСП не исключается и не «невидим»: компонента равна 0, в доводах — «Истории в ФСП нет» (правило 6).

4. Объяснения

Доводы собираются **из тех же компонент**, что и скор, с весом вклада. Берутся 2–4 «за» и 1–2 «против». Примеры — лучший и худший кандидат одной подборки из `validation.md` (сгенерированы кодом, не вручную):

> **77 match** — потребность «Разработчик микросервисов (Junior), java, kubernetes, mysql», релевантен по скрытой истине: да
>
> - Уверенно сдал тест Junior: 27.3/30
> - Java подтверждён тестом
> - Ожидания 80 000 ₽ — в пределах вилки
> - MySQL — косвенно подтверждено тестом − Часть стека не подтверждена тестом

> **38 match** — потребность «Разработчик микросервисов (Junior), java, kubernetes, mysql», релевантен по скрытой истине: нет
>
> - Тест Junior сдан: 19.4/30
> - MySQL — косвенно подтверждено тестом − Нет данных о навыках: Java и Kubernetes − Ожидания 160 000 ₽ — выше вилки на 14%

Свежий сигнал от регулярной задачи. Если за последние 90 дней кандидат решил задачу работодателя на 2–3 из 3, в доводы «за» добавляется «Свежий сигнал: решил задачу работодателя …». Порядок снимка подборки это не меняет, а сам ответ обновляет `lastActiveAt` и через компоненту «свежесть» влияет на следующие подборки (см. `challenges.md`).

LLM только перефразирует (кнопка «Доводы человеческим языком»): один вызов на подборку. Проверка «никаких новых фактов»: в ответе не должно быть чисел, которых нет в исходных доводах, количество доводов должно совпасть. Если проверка не прошла или LLM недоступен, остаются шаблонные формулировки.

5. Снимок и уточнение без потери подборки

- «Подобрать кандидатов» сохраняет `searches.base_snapshot` — упорядоченный список `{profileId, match, near}` на момент поиска.
- Фильтры (грейд, стек и «только подтверждённые тестом», «только с ФСП», порог теста, город, формат, соседний грейд) применяются **поверх снимка**. Снимок не меняется, и это проверено тестом. «Сбросить» = пустые фильтры = исходная подборка.
- Если кандидат после поиска скрыл профиль или отозвал согласие, он выпадает из выдачи (приватность важнее), счётчик показывает это.
- «Поиск по базе» работает тем же движком, но без потребности: специализация, грейды, стек, бюджет, формат, город.
- Избранное (`shortlist`), пагинация, без N+1: пул загружается тремя запросами (категории + профили, навыки, достижения).

6. Порог приглашений и «что, если»

Порог приглашений. Кандидат может задать в профиле минимальный верх вилки. Приглашение ниже порога API не принимает (422 `below_invite_floor`). Порог виден работодателю открыто: в карточке, в форме приглашения (живое предупреждение, кнопка «Отправить» неактивна) и первым доводом «против». В скоринге такой кандидат получает 0 по зарплатной компоненте: пригласить его с этой вилкой нельзя. Обычные площадки здесь тратят время обеих сторон, у нас условия известны до первого сообщения.

«Что, если изменить потребность». Над подборкой показываются подсказки, посчитанные той же формулой (`server/matching/counterfactual.ts`):

Конфликт в топ-10	Подсказка
Ожидания или порог выше верха вилки	Наименьший подъём (+10/20/30%), при котором они входят в вилку, и сколько именно
Предпочитают другой формат	Формат, который полностью подходит большинству из них
Живут в другом городе (офис или гибрид)	Сколько перестанут получать минус без привязки к городу

К каждой подсказке добавляется, сколько новых кандидатов поднимется в верх подборки. Подсказка появляется, только если конфликт реально есть. Так работодатель видит не только «кто подходит», но и «чем вы сами сужаете выбор».

7. Что видит работодатель (правило 2)

Карточка кандидата (`toCard`) **не содержит контактов**. Имя маскируется («Анна Н.») до принятия приглашения или отклика. Тест проверяет, что в JSON ответа подборки нет ни email, ни телефона, ни Telegram, ни фамилии.

8. Качество (из `validation.md`)

На 163 синтетических потребностях: P@5 0.58, P@10 0.53, nDCG@10 0.60 против 0.24 / 0.25 / 0.26 у бейзлайна «совпадение ключевых слов резюме».

5. Регулярные задачи от работодателей

ТЗ: «система периодически предлагает кандидату задачу от работодателя — решить её или предложить подход; это поддерживает профиль в актуальном состоянии и даёт работодателю свежий сигнал о кандидате».

Как это работает

Шаг	Кто	Что происходит
1	Работодатель, `/employer/challenges`	Пишет короткую задачу для специализации и, по желанию, грейда. Выбирает способ проверки: **по эталону** (система сравнивает ответ) или **«предложите подход»** (оценивает сам, 0–3)
2	Система	Кандидату с активной категорией выдаёт **одну** задачу не чаще раза в 14 дней (`CHALLENGE_CONFIG.periodDays`). Выдача ленивая — при входе в кабинет, без cron. Кандидат получает уведомление
3	Кандидат, `/candidate/challenges`	Видит компанию, условие и способ проверки. Отвечает или пропускает без штрафа
4	Система / работодатель	Ответ по эталону проверяется сразу (нормализация: регистр, пробелы, `ё`, десятичная запятая). Открытый ответ попадает работодателю в «Ждут оценки». Работодатель видит замаскированное имя и ответ, **без контактов**
5	Подбор	Хороший результат (≥ 2/3) за последние 90 дней показывается в подборке доводом «Свежий сигнал: решил задачу работодателя …». Ответ обновляет `lastActiveAt` — от него считается компонента «свежесть» скоринга

Почему так

- **Грейд не меняется.** Грейд даёт только квалификационный тест (ответ экспертов FTT.4, VS.5). Задача — сигнал актуальности, а не экзамен.
- **Адресно, а не на всю категорию** (ответ E09). Задачу получает конкретный кандидат, по одной за период.
- **Задача не расходится по рынку.** Из подходящих задач выдаётся наименее показанная (`ORDER BY exposure`), поэтому кандидаты одной категории получают разные задачи. Один кандидат никогда не получает одну задачу дважды (уникальный индекс). Работодатель может закрыть задачу, если считает её «засвеченной».
- **Эталон не уходит на клиент.** `expectedAnswer` читается только в `submitAnswer`, API кандидата его не отдаёт (покрыто тестом).
- **Пропуск без штрафа.** Кандидат не обязан решать: пропуск только сдвигает следующую задачу на период.
- **Вес небольшой.** Задача по эталону проще слить, чем адаптивный тест. Поэтому результат влияет только на свежесть (7% скоринга) и на довод в карточке, но не на категорию и не на порядок снимка подборки.

Где в коде

Что	Где
Таблицы `challenges`, `challenge_assignments`	`server/db/schema.ts`, миграция `drizzle/0007_challenges.sql`
Выдача, проверка, оценка, сигнал	`server/services/challenges.ts`
Параметры	`CHALLENGE_CONFIG` в `server/config.ts`
API	`GET/POST /api/v1/challenges`, `POST /challenges/{id}/close`, `GET /challenges/submissions`, `POST /challenges/submissions/{id}/grade`, `GET /me/challenges`, `POST /me/challenges/{id}/answer`, `POST /me/challenges/{id}/skip`
Довод в подборке	`searchResults` в `server/services/matching.ts`
Тесты	`tests/integration/challenges.test.ts`

В демо-данных у `employer@demo.ru` четыре задачи, два ответа ждут оценки, а `strong@demo.ru` при открытии раздела «Задачи» сразу получает новую.

Развитие

- Плановая выдача по расписанию (cron) и напоминания о незавершённой задаче.
- Задачи с исполнением кода в той же песочнице, что и тест, с параметризацией варианта под кандидата — тогда эталон не сливается.
- Калибровка сложности задач по ответам, как у шаблонов теста, и учёт результата в весах ранжирования.

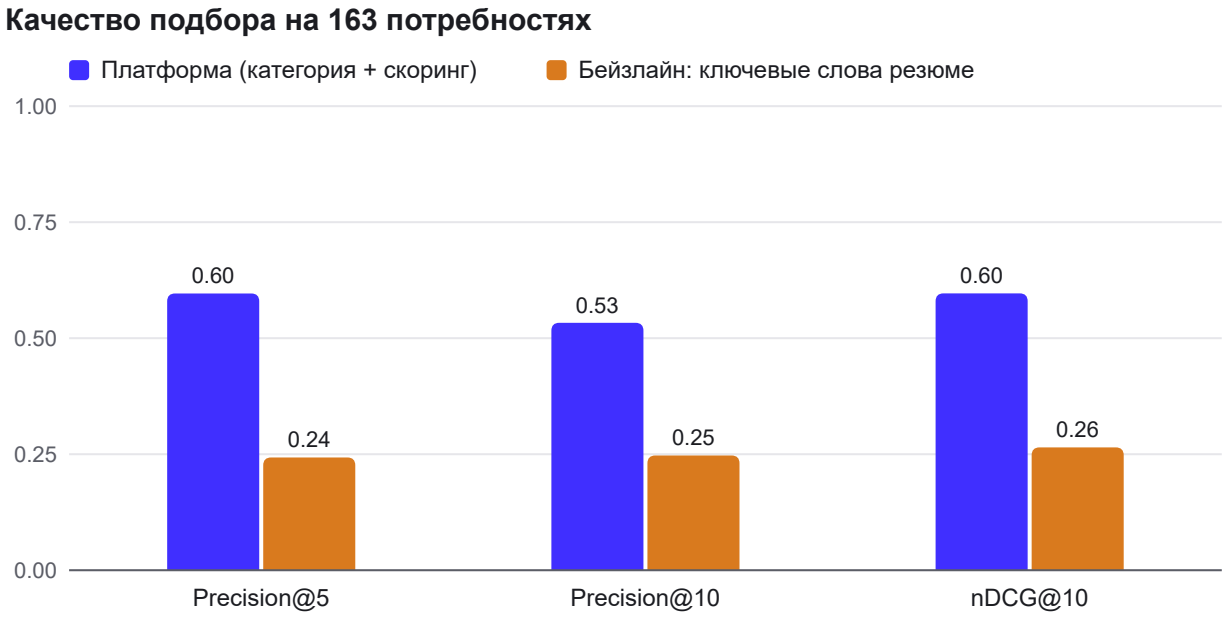
6. Валидация и результаты

Отчёт сгенерирован командой `npm run validate` (2026-10-06), все числа воспроизводимы: генераторы синтетики и симуляций детерминированы (seedrandom). Данные — синтетические, со «скрытой истиной» (истинная способность θ, истинный грейд, реальные навыки, минимально приемлемая зарплата), которая известна только валидатору и никак не используется продуктом.

1. Качество подбора

Популяция: 700 кандидатов, 179 потребностей (163 с хотя бы одним релевантным кандидатом). Разметка «релевантен / нет» — правилами по скрытой истине: та же специализация, **истинный** грейд равен грейду потребности, кандидат реально владеет ≥ 50% стека, его минимальные ожидания ≤ 110% верхней границы вилки, формат совместим.

Метод	Precision@5	Precision@10	nDCG@10
Платформа: жёсткий фильтр по категории из теста + скоринг	**0.60**	**0.53**	**0.60**
Платформа без бонуса ФСП (абляция)	0.61	0.53	0.61
Бейзлайн: совпадение ключевых слов резюме	0.24	0.25	0.26



Главный источник выигрыша — фильтр по категории из теста: самоописание (резюме) не содержит надёжной информации о грейде, ~30% кандидатов в синтетике завышают его, а заявленные навыки содержат «приписки».

Абляция без ФСП даёт чуть более высокий nDCG (0.61 против 0.60): в синтетической разметке релевантность не зависит от достижений, поэтому бонус ФСП здесь — «шум» внутри уже отобранной категории. Это ожидаемо: бонус задуман как способ поднять сильных внутри категории, а не как фактор релевантности; его вес (14%) стоит уточнить на реальных откликах работодателей.

Выборка для ручной проверки

Лучший и худший кандидат подборки для нескольких потребностей — с доводами, которые видит работодатель. Ручная проверка подтверждает, что доводы соответствуют компонентам скоринга и не содержат фактов вне профиля.

Потребность	Match	Релевантен по истине	Доводы «за»	Доводы «против»
Разработчик интерфейсов (Junior), javascript, typescript, redux, react, webpack	68	да	Уверенно сдал тест Junior: 27.9/30; JavaScript и TypeScript подтверждены тестом; Ожидания 90 000 ₽ — в пределах вилки; Заявлен опыт: Redux и Webpack / Vite (не проверено тестом)	Нет данных о навыках: React; Другой город: Ростов-на-Дону
Разработчик интерфейсов (Junior), javascript, typescript, redux, react, webpack	41	нет	JavaScript и TypeScript подтверждены тестом; Тест Junior сдан: 18.5/30; Ожидания 80 000 ₽ — в пределах вилки; Участие в соревнованиях ФСП: 2 подтверждённых результатов	Нет данных о навыках: Redux, React и Webpack / Vite; Предпочитает формат «Удалённо»
React-разработчик (Junior), javascript, typescript, html, nodejs, graphql	71	да	Уверенно сдал тест Junior: 27.9/30; JavaScript и TypeScript подтверждены тестом; Ожидания 90 000 ₽ — в пределах вилки; Заявлен опыт: HTML и GraphQL (не проверено тестом)	Другой город: Ростов-на-Дону
React-разработчик (Junior), javascript, typescript, html, nodejs, graphql	46	да	JavaScript и TypeScript подтверждены тестом; Тест Junior сдан: 18.5/30; Ожидания 140 000 ₽ — в пределах вилки; Node.js — косвенно подтверждено тестом	Нет данных о навыках: HTML и GraphQL; Предпочитает формат «Удалённо»
Разработчик микросервисов (Junior), java, kubernetes, mysql	77	да	Уверенно сдал тест Junior: 27.3/30; Java подтверждён тестом; Ожидания 80 000 ₽ — в пределах вилки; MySQL — косвенно подтверждено тестом	Часть стека не подтверждена тестом
Разработчик микросервисов (Junior), java, kubernetes, mysql	38	нет	Тест Junior сдан: 19.4/30; MySQL — косвенно подтверждено тестом	Нет данных о навыках: Java и Kubernetes; Ожидания 160 000 ₽ — выше вилки на 14%
Тестировщик (Junior), testdesign, postman, selenium, linux, python, pytest	75	да	Уверенно сдал тест Junior: 26.9/30; Тест-дизайн, Postman и Python подтверждены тестом; Ожидания 130 000 ₽ — в пределах вилки; Призёр ФСП: 2 место, Всероссийский хакатон ФСП (региональный этап, 2025)	Часть стека не подтверждена тестом
Тестировщик (Junior), testdesign, postman, selenium, linux, python, pytest	46	нет	Тест-дизайн, Postman и Python подтверждены тестом; Тест Junior сдан: 18.7/30; Ожидания 100 000 ₽ — в пределах вилки; pytest — косвенно подтверждено тестом	Нет данных о навыках: Selenium и Linux; Предпочитает формат «Гибрид»

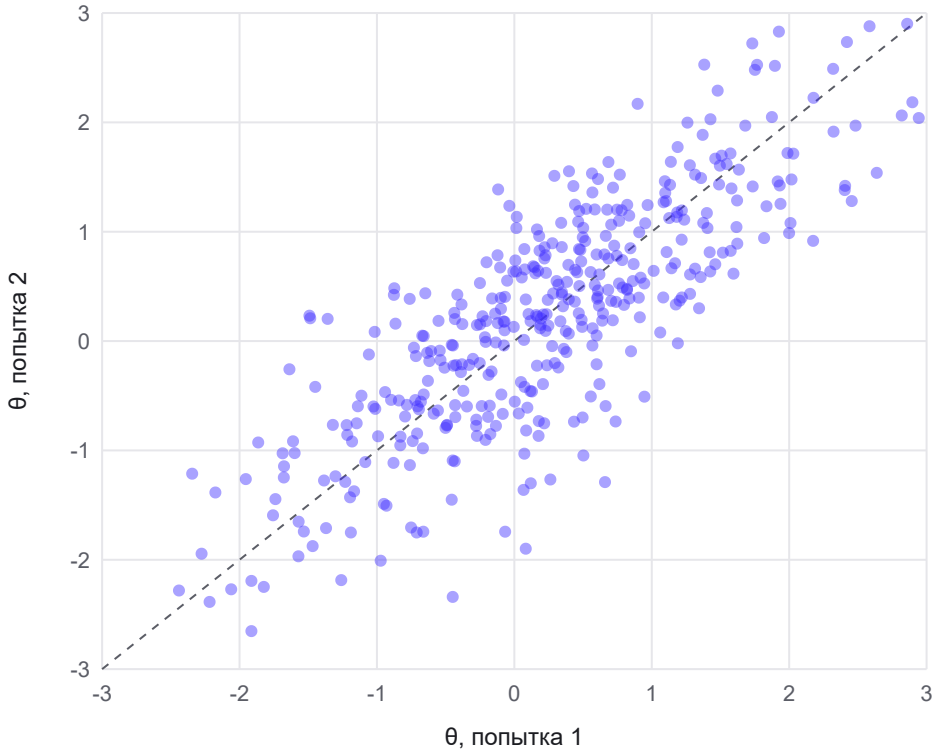
Потребность	Match	Релевантен по истине	Доводы «за»	Доводы «против»
Разработчик интерфейсов (Junior), javascript, typescript, nodejs, jest	73	да	Уверенно сдал тест Junior: 27.9/30; JavaScript и TypeScript подтверждены тестом; Ожидания 90 000 ₽ — в пределах вилки; Node.js — косвенно подтверждено тестом	Предпочитает формат «Гибрид»
Разработчик интерфейсов (Junior), javascript, typescript, nodejs, jest	50	да	JavaScript и TypeScript подтверждены тестом; Тест Junior сдан: 18.8/30; Ожидания 140 000 ₽ — в пределах вилки; Node.js — косвенно подтверждено тестом	Нет данных о навыках: Jest; Предпочитает формат «Гибрид»
Backend-разработчик (Junior), python, rest, microservices	72	нет	Уверенно сдал тест Junior: 26.7/30; Python подтверждён тестом; Заявлен опыт: REST API и Микросервисы (не проверено тестом); Ожидания 80 000 ₽ — в пределах вилки	Подтверждённых призовых мест в ФСП нет
Backend-разработчик (Junior), python, rest, microservices	36	нет	Python подтверждён тестом; Тест Junior сдан: 18.8/30	Нет данных о навыках: REST API и Микросервисы; Ожидания 170 000 ₽ — выше вилки на 13%

2. Надёжность теста (тест-ретест)

400 симулированных кандидатов (θ ~ N(0.1, 1.05)) дважды проходят адаптивный тест своего истинного грейда на банке Backend. Варианты заданий разные (другой seed попытки → другие параметры, данные и порядок отбора из top-K).

Показатель	Значение
Корреляция θ между попытками	**0.81**
Корреляция оценки θ с истинной θ	0.89
Совпадение решения «сдал / не сдал»	**78.5%**
Совпадение итога (уверенно / сдал / не сдал)	65.0%
Средняя длина теста	16.0 заданий (мин. 10, макс. 18)

Тест-ретест: θ в двух попытках с разными вариантами

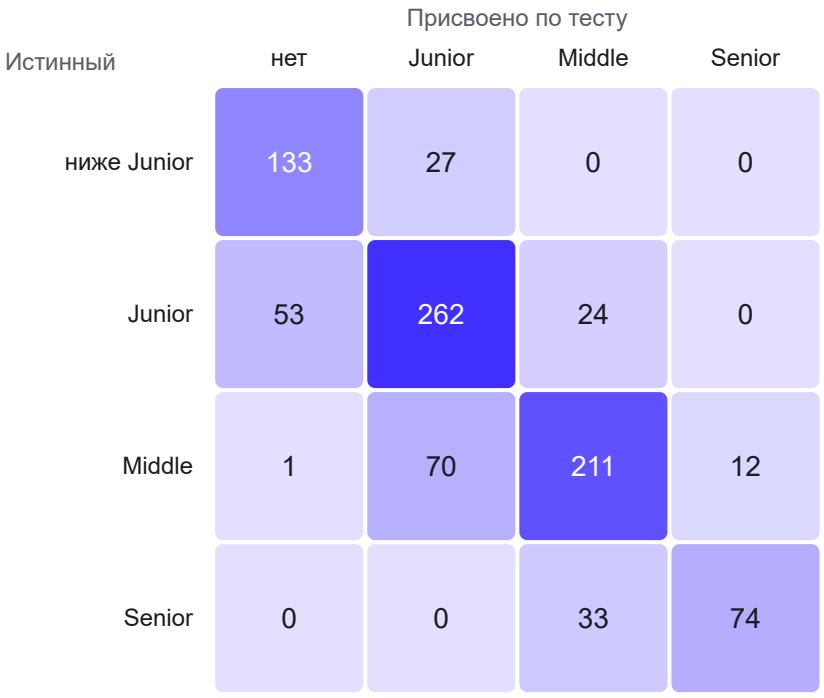


3. Различающая способность: присвоение грейда

900 кандидатов: 70% выбирают свой истинный грейд, остальные ошибаются на ступень (чаще — вверх). Дальше они действуют по правилам пересдачи: после провала — сразу грейд ниже, после уверенного результата (≥ 25/30) — сразу грейд выше. Категория = наивысший сданный грейд.

Показатель	Значение
Итог совпал с истинным уровнем (включая «нет категории» для θ ниже порога Junior)	**75.6%**
Ошибка на одну ступень	24.3%
Категория выше истинного грейда (самое дорогое для работодателя)	7.0%
Среднее число попыток на кандидата	1.36
Средняя длина попытки (CAT)	15.8 заданий
Причины остановки	SE < порога: 13.2%; максимум заданий: 57.2%; решение очевидно (

Истинный уровень × присвоенная категория



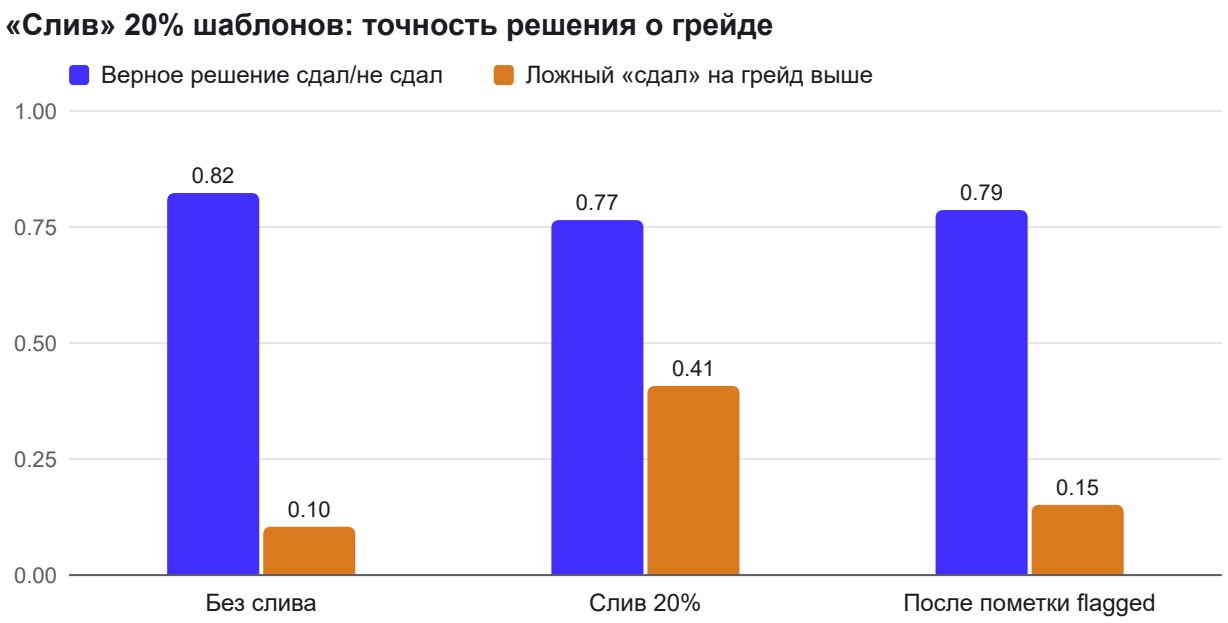
Строки — истинный уровень (θ ниже порога Junior — категория не положена), столбцы — присвоенная категория («нет» — ни один тест не сдан). Ошибки сосредоточены у границ грейдов: кандидат с θ рядом с порогом с вероятностью ~50% окажется по любую сторону — это свойство любой процедуры с порогами, а не адаптивности.

4. Устойчивость к утечке заданий

Симулируем «слив» 5 из 27 шаблонов Backend (20%): на слитых заданиях кандидат отвечает верно с вероятностью 0.95 независимо от способности. Это худший случай: утёк сам шаблон, а не отдельный вариант (варианты параметризованы, и слив одного варианта почти не помогает). Половина когорты пробует грейд выше своего истинного — там слив опаснее всего.

Сценарий	Верное решение «сдал / не сдал»	Ложный «сдал» при попытке грейда выше	Смещение θ	RMSE θ
Без слива	82.3%	10.4%	-0.01	0.45
Слив 20% шаблонов	76.5%	40.8%	0.46	0.72
После калибровки: слитые помечены flagged и сняты с оценки	78.7%	15.1%	0.01	0.49

Детектор дрейфа (доля верных выше прогноза IRT на ≥ 15 п.п. при z ≥ 3 и ≥ 30 ответах) пометил **5** шаблонов, из них слитых — **5 из 5** (полнота 100.0%, точность 100.0%). На чистых данных ложных пометок: 0.



5. Ловушки и индекс доверия

По 500 симулированных попыток каждого типа. Индекс доверия — 100 минус сумма сигналов (маркер, двойник, вставки, уходы со вкладки, выход из полноэкранного режима, быстрые верные ответы на сложных заданиях, person-fit); уровни: high ≥ 70, medium 40–69, low < 40.

Тип	high (категория)	medium (с пометкой)	low (перепроверка, без категории)
Честные	100.0%	0.0%	0.0%
Копируют условие в ИИ	0.0%	0.0%	100.0%
Фото экрана → ИИ	3.8%	0.4%	95.8%

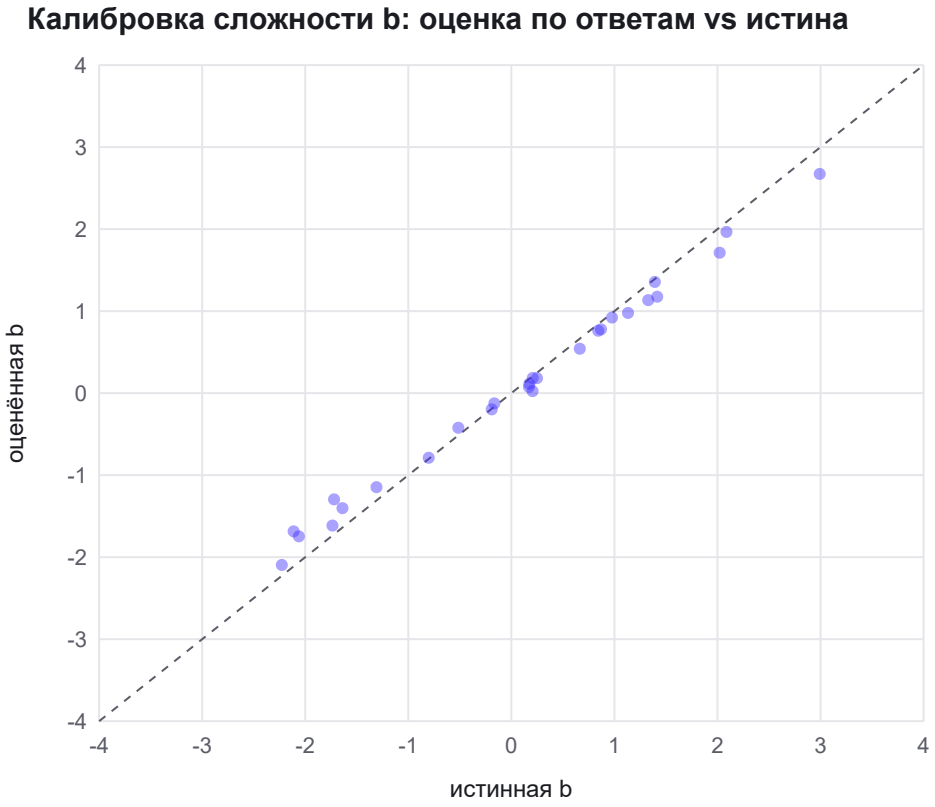


Допущения симуляции (честно): ассистент, получивший скрытую инструкцию, использует маркер в 60% кодовых заданий; при копировании DOM-текста значение-двойник попадает в расчёт в 90% случаев; по фото экрана микротекст считывается и исполняется в 25% случаев. Реальные доли нужно измерить на пилоте — индекс настраивается весами в `server/testing/trust.ts`. Ложные срабатывания на честных — это доля честных с уровнем low; автоматических блокировок нет в любом случае.

6. Калибровка параметров заданий

Стартовые параметры IRT (a, b) — экспертные. Чтобы проверить процедуру калибровки (`npm run calibrate`), задаём «истинные» параметры, отличающиеся от экспертных (шум σ_b = 0.45, σ_a = 0.25), симулируем 3000 прохождений и оцениваем a, b логистической регрессией по оценкам θ.

Показатель	Значение
RMSE экспертной b относительно истинной	0.47
RMSE откалиброванной b	**0.19**
Корреляция откалиброванной и истинной b	1.00



В проде калибровка идёт на реальных ответах: счётчики `answered/correct/expected` копятся по каждому шаблону, детектор утечки срабатывает онлайн, а пересчёт a, b выполняется периодически тем же кодом (`server/testing/calibration.ts`).

Как воспроизвести

```
npm run validate   # пишет docs/validation.md и docs/img/*.svg
```

Время прогона на ноутбуке: ~31 с.

7. Интеграция с ФСП

API ФСП для кейса не предоставлен, поэтому платформа работает через интерфейсы. Реальную интеграцию можно подключить заменой реализаций, не трогая бизнес-логику.

Что реализовано в прототипе

Компонент	Файл	Статус
Интерфейс реестра `FspRegistry.lookup(fspId)`	`server/fsp/registry.ts`	готов
`MockFspRegistry` — детерминированный синтетический реестр	там же	готов
Привязка ФСП ID в профиле и импорт достижений (`source = fsp_registry`, `trust = high`)	`server/services/candidate.ts → linkFsp`	готов, покрыт тестами
Деградация при недоступности реестра (503, ранее импортированные данные сохраняются)	там же	готов
Вход через «ФСП ID» (OIDC через Keycloak), флаг `AUTH_KEYCLOAK_ENABLED`	`server/auth/index.ts`, `infra/keycloak/realm.json`, профиль `keycloak` в docker-compose	готов, по умолчанию выключен

Поведение мока для демо:

- ID формата `FSP-123456`;
- окончание `000` — человека нет в реестре (404);
- окончание `999` — реестр недоступен (503);
- остальные ID — 1–5 достижений, детерминированных по ID.

Модель данных реестра (как у ФСП)

```
Человек (fspId, ФИО, регион, спортивный разряд, рейтинг)
 └ Соревнование (название, дисциплина: алгоритмическое | продуктовое | ИБ | робототехника | БАС)
     └ Этап (региональный → окружной → финал, год)
         └ Результат (место | позиция в рейтинге, команда)
```

Маппинг полей

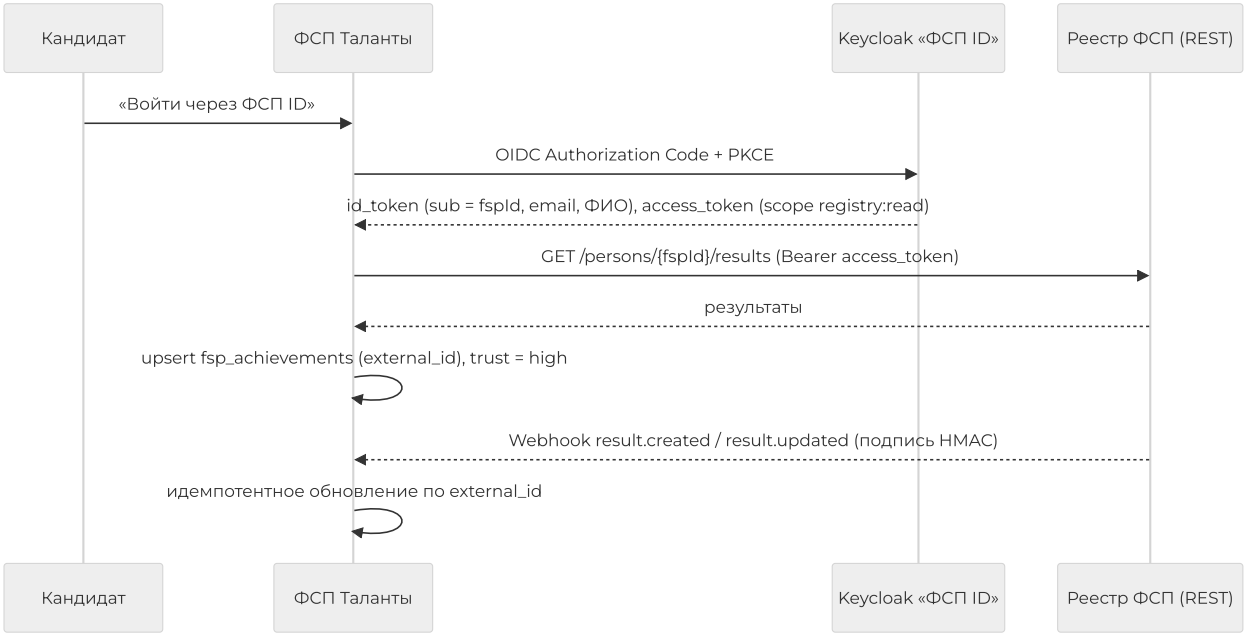
Реестр ФСП	Платформа (`fsp_achievements`)
ID результата	`external_id` (уникален в паре с профилем — повторный импорт идемпотентен)
Соревнование	`competition`
Дисциплина	`discipline`
Этап	`stage` (`regional` / `district` / `final`)
Год	`year`
Место	`place` (null — участие без места)

Реестр ФСП	Платформа (`fsp_achievements`)
Позиция в рейтинге	`rank`
Спортивный разряд	`sports_category`
Команда, рейтинг, регион	`evidence` (JSONB)
—	`source = fsp_registry` , `trust_level = high`

Уровни доверия к достижениям

Источник	Как подтверждается	Вес в скоринге
`fsp_registry`	Импорт из реестра по ФСП ID	1.0
`organizer`	Сертификат организатора без сигналов риска в метаданных (PDF: Creator/Producer, даты, инкрементальные сохранения, подпись; изображения: EXIF)	0.8
`repo`	Репозиторий GitHub: кандидат среди авторов, коммиты в окне мероприятия, не форк	0.6
`self`	Заявлено кандидатом (или сертификат с сигналами риска)	0.2

Схема будущей реальной интеграции



1. **Аутентификация.** Провайдер Keycloak в Auth.js уже подключён. Для продакшена:
   - заменить `AUTH_KEYCLOAK_ISSUER` на реальный realm ФСП;
   - добавить claim `fsp_id` (в демо-realm он лежит в атрибутах пользователя) и сохранять его в `candidate_profiles.fsp_id` при первом входе.
2. **Чтение реестра.** Реализация `HttpFspRegistry implements FspRegistry`:
   - REST с токеном пользователя (scope `registry:read`) или сервисным токеном;
   - таймаут 5 с, ретрай с экспоненциальной задержкой, кеш ответа на 24 часа.
3. **Обновления.** Вебхуки реестра (`result.created` , `result.updated` , `person.merged`) на `/api/v1/integrations/fsp/webhook` :

    - проверка HMAC-подписи;
    - идемпотентность по `external_id`;
    - если вебхуки недоступны — ночная сверка или ленивое обновление при открытии профиля (без cron на бесплатном Vercel).
4. **Недоступность реестра.** Ранее импортированные достижения остаются и учитываются. Новая привязка возвращает 503 с понятным текстом. Достижения можно временно подтвердить сертификатом или репозиторием (доверие ниже), после восстановления реестра — переимпорт.
5. **Конфликты.** Один ФСП ID — один профиль (проверяется в `linkFsp`, ответ 409). Слияние персон в реестре обрабатывается по вебхуку `person.merged`.

Keycloak локально

```
AUTH_KEYCLOAK_ENABLED=true docker compose --profile keycloak up -d
# realm "fsp" импортируется из infra/keycloak/realm.json
# пользователь: fsp-candidate / Demo12345, клиент fsp-talents
```

При входе через OIDC пользователь создаётся с ролью «кандидат» и подтверждённым email (`upsertOidcUser`). Согласия на обработку ПДн собираются при первом визите в кабинет.

> Проверено юнит-логикой и сборкой. Полный OIDC-обмен с поднятым Keycloak в среде разработки прототипа запустить не удалось (Docker Hub ограничивал загрузку образов), поэтому профиль `keycloak` стоит проверить локально.

8. API

- Интерактивная документация: `/api/docs` (Swagger UI).
- Спецификация OpenAPI 3.1: `/api/openapi.json`, выгрузка в репозитории — `openapi.json` (`npm run openapi`).

Спецификация строится из **тех же Zod-схем**, что валидируют запросы (`server/schemas/*`), поэтому документация не расходится с кодом.

Соглашения

Что	Как
Префикс	`/api/v1`
Аутентификация	Сессия Auth.js (httpOnly cookie). Вход — `/login`; для скриптов — `POST /api/auth/callback/credentials` с CSRF-токеном
Формат ошибок	`{ "error": { "code", "message", "details?" } }`
401 / 403	Нет сессии / нет прав по роли
404	Не найдено **или чужое** (не раскрываем существование)
409	Недопустимый переход статуса, дубль, кулдаун пересдачи
422	Ошибка валидации — в т. ч. отсутствие вилки зарплаты
429	Rate limit (вход, регистрация, приглашения, LLM-эндпоинты)

Группы эндпоинтов

Группа	Основные пути
Аккаунт	`POST /auth/register`, `POST /auth/resend`
Кандидат	`GET/PUT /me/profile`, `PUT /me/contacts`, `PUT /me/skills`, `PUT /me/visibility`, `POST /me/consents`, `GET /me/access-log`, `POST /me/resume`, `GET /me/resume.pdf`, `POST /me/fsp`, `POST /me/achievements`, `POST /me/certificates`, `POST /me/repo-check`, `DELETE /me`
Тестирование	`GET /tests`, `POST /tests/attempts`, `GET /tests/attempts/{id}`, `POST …/items/{itemId}/hidden`, `POST …/items/{itemId}/answer`, `POST …/events`, `POST …/finish`
Подбор	`POST /needs/parse`, `GET/POST /needs`, `POST /needs/{id}/search`, `GET/POST /searches`, `GET /searches/{id}?фильтры`, `POST /searches/{id}/rephrase`, `GET /shortlist`, `PUT/DELETE /shortlist/{profileId}`
Приглашения	`GET/POST /invitations`, `GET /invitations/{id}`, `POST /invitations/{id}/decision`, `POST /invitations/{id}/withdraw`, `GET /candidates/{profileId}`
Вакансии	`GET/POST /vacancies`, `POST /vacancies/{id}/apply`, `POST /vacancies/{id}/close`, `GET /vacancies/{id}/applications`, `PATCH /applications/{id}`
Модерация	`GET /admin/attempts`, `GET /admin/attempts/{id}`, `POST /admin/attempts/{id}/review`, `GET /admin/templates`, `PATCH /admin/templates/{id}`, `GET /admin/plagiarism`
Регулярные задачи	`GET/POST /challenges`, `POST /challenges/{id}/close`, `GET /challenges/submissions`, `POST /challenges/submissions/{id}/grade`, `GET /me/challenges`, `POST /me/challenges/{id}/answer`, `POST /me/challenges/{id}/skip`

Группа	Основные пути
Чаты	`GET /chats`, `GET/POST /chats/{id}/messages`, `GET /me/unread`
Видеоинтервью	`POST /chats/{id}/interviews`, `GET /interviews`, `GET /interviews/{id}`, `POST /interviews/{id}/token`, `PUT /interviews/{id}/code`, `POST /interviews/{id}/cancel`, `POST /interviews/{id}/finish`
Уведомления	`GET /notifications`, `POST /notifications/read`

Чего в API нет намеренно: кандидат не может получить чужой профиль ни по одному пути, а эндпоинт текущего задания никогда не возвращает эталоны, решатели и следующие задания.

9. Безопасность и 152-ФЗ

Жёсткие правила ТЗ и где они проверяются

#	Правило	Реализация	Тесты
1	Вилка зарплаты обязательна, целые рубли, 0 < от ≤ до; без неё 422	`withSalaryRange` в Zod-схемах потребности, приглашения, вакансии + повторная проверка в сервисе приглашений	`tests/unit/salary.test.ts`, `invitations.test.ts`, `vacancies.test.ts`, e2e (отправка без вилки отклоняется)
2	Контакты скрыты до принятия приглашения или отклика	Отдельная таблица `candidate_contacts`; сериализатор карточки без контактов; `hasContactAccess` в сервисе; маскирование имени; аудит раскрытия и каждого чтения	`matching.test.ts` (в JSON подборки нет контактов), `invitations.test.ts`, `vacancies.test.ts`, e2e
3	Статусы `sent → viewed → accepted / declined` + `withdrawn`, `expired`; недопустимые переходы — 409; история видна обеим сторонам	`server/invitations/fsm.ts`, оптимистичная блокировка по статусу, `invitation_events`	`fsm.test.ts`, `invitations.test.ts`
4	Кандидат видит только своё	Проверка владельца в каждом сервисе, чужое → 404, нет эндпоинтов чужих профилей	`testing.test.ts`, `invitations.test.ts`, `candidate.test.ts`
5	Выдача — по категориям из теста	`loadPool`: только активная категория; навыки из резюме помечаются «заявлено»	`matching.test.ts` (кандидат без категории с навыками в резюме отсутствует)
6	Без истории в ФСП — просто нет бонуса	Компонента `fsp = 0`, довод «Истории в ФСП нет»	`score.test.ts`
7	Понижение не принудительное; кулдаун пересдачи	`eligibility.ts` (14 дней тот же грейд, ниже — сразу, выше — после ≥ 25/30); провал не снимает категорию	`eligibility.test.ts`, `testing.test.ts`

Аутентификация и сессии

- Пароли: **argon2id** (19 МиБ, t = 2, p = 1). При неизвестном email выполняется фиктивная проверка, чтобы по времени ответа нельзя было определить, есть ли аккаунт.
- Подтверждение email: одноразовый токен (в БД хранится только SHA-256), срок 48 часов. До подтверждения вход запрещён.
- Сессии Auth.js v5 — JWT в httpOnly cookie (secure в проде). CSRF — встроенная защита Auth.js для форм входа. Server actions Next.js проверяют Origin.
- Роли проверяются в `middleware.ts` (разделы) и **в сервисах**: роль и признак удаления перечитываются из БД на каждом запросе.
- Rate limit: вход (10 за 15 мин на email), регистрация (10 в час на IP), приглашения (60 в час и 30 в сутки на работодателя), LLM-эндпоинты (30 в час). Upstash Ratelimit при наличии ключа; иначе in-memory. **Ограничение:** на serverless in-memory лимит действует в пределах инстанса, поэтому суточный лимит приглашений считается по БД.

- Заголовки: `X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options: DENY`, `Permissions-Policy`; COOP/COEP только на странице теста.

152-ФЗ

- Явные согласия с **версией и датой** (`consents`): обработка ПДн и политика конфиденциальности обязательны, рассылка — по желанию. Отзыв согласия на обработку ПДн убирает кандидата из подборок.
- Политика обработки ПДн — отдельная страница `/privacy`.
- **Минимизация:**
  - работодатель видит маскированное имя и не видит контактов до согласия кандидата;
  - журнал попытки хранит длину вставки, но не содержимое буфера обмена;
  - в логах приложения нет ПДн: почтовый логгер скрывает адресата, ошибки Auth.js не логируют email;
  - **в LLM не уходят ФИО и контакты** (`server/llm/redact.ts`). ФИО, email, телефон, Telegram и ссылки на профили извлекаются из резюме локальными правилами и сразу пишутся в защищённую таблицу. Модель получает текст с метками `[ИМЯ]`, `[EMAIL]`, `[PHONE]`, `[TELEGRAM]`, `[LINK]`, `[ДАТА РОЖДЕНИЯ]` и не возвращает эти поля вовсе (их нет в схеме ответа). Из текста вакансии так же убираются контакты рекрутера. Перефразирование доводов получает только сами доводы, без имени. Это минимизация, а не полная анонимизация: свободный текст может содержать косвенные сведения (места работы, даты), поэтому модель выбирается из рекомендованных организаторами, а без ключа все функции работают на правилах.
- **Журнал доступа к контактам** (`audit_log`: `contacts.disclosed`, `contacts.viewed`) доступен кандидату в разделе «Приватность».
- **Удаление аккаунта с анонимизацией:**
  - email заменяется, пароль удаляется;
  - ФИО, город, «о себе», ФСП ID обнуляются;
  - контакты, навыки, резюме и достижения удаляются;
  - открытые приглашения истекают, согласия отзываются;
  - обезличенная статистика тестов остаётся для калибровки.
- Кандидат заранее предупреждён о журнале попытки: подтверждение перед стартом теста и правила на странице выбора. Автоматических банов нет — решение по подозрительной попытке принимает человек.

Секреты

Все ключи передаются только через переменные окружения (`.env.example` описывает каждую). Ключ RouterAI используется только на сервере (`server/llm`). Каталог `.env*` в `.gitignore`.

Файлы

- PDF-резюме и сертификаты: проверка сигнатуры файла (`%PDF-`, PNG, JPEG), лимит 4 МБ.
- Хранение в Vercel Blob со случайным суффиксом; URL не отдаётся другим пользователям.
- **Ограничение:** Blob-объекты публичны по неугадываемому URL — для продакшена нужен приватный бакет или подписанные ссылки.

Защита от фиктивных вакансий и недобросовестных работодателей

Работает без ручной модерации каждой вакансии: сигналы считаются из поведения самой компании (`server/invitations/reputation.ts`, `server/services/reputation.ts`).

Сигнал	Что происходит
Компания новая (меньше 10 приглашений)	Лимит 15 приглашений в сутки, у кандидата пометка «Новая компания на платформе»
Кандидаты принимают ≥ 30% приглашений, жалоб нет	Лимит 50 в сутки, пометка «Кандидаты часто принимают приглашения»
≥ 3 жалоб за 30 дней или подтверждённая жалоба	Статус риска: лимит 5 в сутки, кандидат видит причину
Почти все приглашения отклоняются (< 5% из ≥ 20)	Статус риска — признак массовой рассылки
Больше половины приглашений отозвано	Статус риска — признак «сбора резюме»
Компания проверена	+10 к лимиту, но не снимает статус риска

Кандидат в каждом приглашении видит репутацию компании: долю принятых приглашений, долю ответов, дату регистрации. Кнопка «Пожаловаться» (фиктивная вакансия, несоответствие условий, спам, некорректное поведение) отклоняет открытое приглашение и попадает модератору в `/admin/reports`. Модератор подтверждает или отклоняет жалобу; автоматических блокировок нет. Обязательная вилка зарплаты и порог приглашений кандидата дополнительно не дают «разведывать» зарплаты бесплатно.

Развитие после MVP: верификация компании по ИНН через ЕГРЮЛ (поле `inn` уже есть), проверка домена почты работодателя, интеграция с ATS через вебхуки статусов приглашений (`invitation_events` уже хранит всю историю переходов).

10. Интеграция с ATS (концепция)

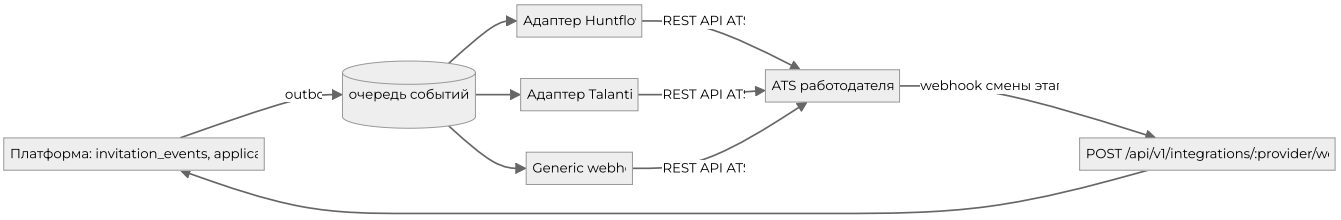
За рамками MVP. Цель (ответ экспертов FTT.5): платформа встраивается в существующий процесс найма работодателя, а не требует вручную дублировать данные между системами.

Что синхронизируем

Сущность платформы	Объект в ATS (Huntflow, Talantix, Potok, SAP SF и др.)	Направление
Потребность `needs` / вакансия `vacancies`	Вакансия	ATS → платформа (импорт), платформа → ATS (создание)
Принятое приглашение / принятый отклик	Кандидат на вакансии + резюме PDF	платформа → ATS
Статусы `invitation_events`, `applications.status`	Этапы воронки	в обе стороны
Результат теста и категория	Комментарий / кастомное поле кандидата	платформа → ATS
Отказ, оффер, выход на работу	Этап «отказ / оффер / нанят»	ATS → платформа (для качества подбора)

Контакты передаются в ATS только после согласия кандидата: после принятия приглашения или собственного отклика. До этого ATS получает обезличенную карточку, так работает правило ТЗ о раскрытии контактов.

Архитектура



- **Outbox-паттерн.** Каждое изменение статуса уже пишется в `invitation_events`. Отдельная таблица `integration_outbox` получает событие в той же транзакции. Воркер доставляет его адаптеру с повторами и идемпотентным ключом. Событие не теряется, даже если ATS недоступна.
- **Адаптер на каждую ATS** реализует общий интерфейс `pushCandidate`, `pushStage`, `importVacancies`, `parseWebhook`. Маппинг этапов хранится в настройках работодателя: «принято» → «Первичное интервью».
- **Generic webhook** для ATS без готового адаптера: работодатель указывает URL, платформа шлёт подписанные (HMAC) события в стандартном JSON.
- **Подключение**: OAuth 2.0 там, где ATS его поддерживает, иначе API-ключ. Токены шифруются, доступ к ним — только у сервиса интеграций.
- **Обратная связь в подбор**: исходы из ATS («оффер», «нанят», «отказ после интервью») — разметка для обучения весов ранжирования (learning-to-rank) с сохранением объяснимости.

Что уже есть в MVP

- Полная история статусов приглашений в `invitation_events` — готовый источник событий.
- Стандартизированный PDF-профиль кандидата (`GET /api/v1/me/resume.pdf`) — вложение для карточки в ATS.
- OpenAPI-описание всего API (`/api/docs`) — работодатель может забирать данные сам.

152-ФЗ

Передача персональных данных в ATS — это передача третьему лицу (работодателю-оператору). Поэтому нужны отдельная строка в согласии кандидата и реестр получателей. В журнал доступа кандидата (`GET /me/access-log`) попадает и выгрузка в ATS.

11. Библиотеки и версии

Сгенерировано скриптом `npm run docs:libs` из `package.json` и установленных пакетов.

Пакет	Версия	Лицензия	Тип	Назначение
`@asteasolutions/zod-to-openapi`	9.1.0	MIT	runtime	OpenAPI 3.1 из Zod-схем
`@codemirror/commands`	6.11.1	MIT	runtime	CodeMirror
`@codemirror/lang-javascript`	6.2.5	MIT	runtime	Подсветка JS
`@codemirror/lang-python`	6.2.1	MIT	runtime	Подсветка Python
`@codemirror/lang-sql`	6.10.0	MIT	runtime	Подсветка SQL
`@codemirror/language`	6.12.4	MIT	runtime	CodeMirror
`@codemirror/state`	6.7.6	MIT	runtime	CodeMirror
`@codemirror/view`	6.43.13	MIT	runtime	CodeMirror
`@lezer/highlight`	1.2.5	MIT	runtime	CodeMirror (подсветка синтаксиса)
`@livekit/components-react`	2.9.24	Apache-2.0	runtime	
`@livekit/components-styles`	1.2.0	Apache-2.0	runtime	
`@node-rs/argon2`	2.2.1	MIT	runtime	Хеширование паролей (argon2id)
`@react-pdf/renderer`	4.9.0	MIT	runtime	Генерация PDF-резюме
`@upstash/ratelimit`	2.2.0	MIT	runtime	Rate limit (если задан Upstash)
`@upstash/redis`	1.39.0	MIT	runtime	Rate limit (если задан Upstash)
`@vercel/analytics`	2.0.1	MIT	runtime	
`@vercel/blob`	2.8.0	Apache-2.0	runtime	Хранилище файлов на Vercel
`codemirror`	6.0.2	MIT	runtime	Редактор кода в тесте
`drizzle-orm`	0.45.3	Apache-2.0	runtime	ORM для PostgreSQL
`exifr`	7.1.3	MIT	runtime	EXIF изображений сертификатов
`livekit-client`	2.22.4	Apache-2.0	runtime	
`livekit-server-sdk`	2.19.1	Apache-2.0	runtime	
`next`	15.5.27	MIT	runtime	Фреймворк: UI (App Router) и API (route handlers)
`next-auth`	5.0.0-beta.32	ISC	runtime	Аутентификация (Auth.js v5): пароль, роли, Keycloak OIDC
`nodemailer`	8.0.11	MIT-0	runtime	Почта через SMTP (Mailpit локально)
`openai`	7.28.0	Apache-2.0	runtime	Клиент RouterAI (OpenAI-совместимый API)

Пакет	Версия	Лицензия	Тип	Назначение
`pdf-lib`	1.17.1	MIT	runtime	Метаданные и структура PDF-сертификатов
`postgres`	3.4.9	Unlicense	runtime	Драйвер PostgreSQL
`react`	19.3.0	MIT	runtime	UI
`react-dom`	19.3.0	MIT	runtime	UI
`resend`	6.32.0	MIT	runtime	Почта на Vercel
`seedrandom`	3.0.5	MIT	runtime	Детерминированный RNG генераторов заданий
`sql.js`	1.14.2	MIT	runtime	SQLite/WASM: эталон SQL-заданий на сервере и песочница в браузере
`unpdf`	1.8.1	MIT	runtime	Извлечение текста из PDF-резюме
`zod`	4.6.5	MIT	runtime	Валидация входных данных
`@eslint/eslintrc`	3.3.7	MIT	dev	
`@playwright/test`	1.63.0	Apache-2.0	dev	Сквозные тесты
`@tailwindcss/postcss`	4.3.3	MIT	dev	Стили
`@types/node`	26.6.4	MIT	dev	
`@types/nodemailer`	8.0.2	MIT	dev	
`@types/react`	19.3.0	MIT	dev	
`@types/react-dom`	19.3.0	MIT	dev	
`@types/seedrandom`	3.0.8	MIT	dev	
`@types/sql.js`	1.4.11	MIT	dev	
`@vitest/coverage-v8`	5.0.3	MIT	dev	
`dotenv`	18.0.5	BSD-2-Clause	dev	
`drizzle-kit`	0.31.11	MIT	dev	Миграции схемы
`eslint`	9.39.5	MIT	dev	Линтер
`eslint-config-next`	15.5.27	MIT	dev	
`postcss`	8.5.29	MIT	dev	
`prettier`	3.9.9	MIT	dev	Форматирование
`prettier-plugin-tailwindcss`	0.8.1	MIT	dev	
`tailwindcss`	4.3.3	MIT	dev	Стили
`tsx`	4.23.15	MIT	dev	Запуск TS-скриптов (seed, validate)
`typescript`	6.0.3	Apache-2.0	dev	Язык
`vitest`	5.0.3	MIT	dev	Юнит- и интеграционные тесты

Загружаются в браузере с CDN jsDelivr: Pyodide 314.0.7 (Python в Web Worker), sql.js 1.14.2 (SQLite/WASM), swagger-ui-dist 5 (страница /api/docs).

Шрифты: Golos Text и JetBrains Mono (SIL OFL 1.1; файлы лицензий — `server/pdf/fonts/`).

12. Ограничения и план развития

Честные ограничения

Область	Ограничение	Что с этим делать
Проверка кода	Код выполняется в браузере: скорость и память не проверяются, исполнение технически можно вынести за пределы браузера. Верный ответ всё равно требует решить задачу на свежих скрытых входах	Серверная песочница (Judge0 / Piston на VPS) для задач с лимитами времени и памяти; интерфейс проверки уже готов
Точность грейда	На синтетике ~76% совпадений с истинным уровнем, ошибки на одну ступень у границ грейдов; CAT почти всегда использует 18 заданий	Расширять банк (больше информативных заданий у каждого порога), калибровать на реальных ответах, ввести «пограничную зону» с перепроверкой
Калибровка	Стартовые a, b — экспертные; калибровка проверена на синтетике	Копить реальные ответы; пересчёт `npm run calibrate` по расписанию вне Vercel
Ловушки	Доли срабатывания ИИ-ассистентов в валидации — допущения	Измерить на пилоте и настроить веса в `server/testing/trust.ts`
Полноэкранный режим	iPhone Safari не поддерживает Fullscreen API для страницы — тест идёт без него (это не штрафуется)	Для мобильных — короче тест и больший вес сигналов фокуса
Rate limit	Без Upstash лимиты in-memory действуют на один serverless-инстанс	Подключить Upstash Redis (бесплатный тариф)
Файлы	Vercel Blob — публичные объекты по неугадываемому URL	Приватное хранилище или подписанные ссылки
Реестр ФСП	Мок с той же структурой данных	`HttpFspRegistry` + вебхуки (см. `fsp-integration.md`)
Keycloak	Профиль docker-compose и провайдер готовы; полный OIDC-обмен в среде разработки не прогнан (Docker Hub ограничивал загрузку образов)	Проверить `docker compose --profile keycloak up` локально
GitHub	Без OAuth-привязки логин заявляется кандидатом	OAuth GitHub App, проверка `viewer.login`
Сертификаты	Подпись PDF обнаруживается, но цепочка сертификатов не проверяется	Проверка подписи (PKCS#7) и QR/ID организатора через API организаторов
Семантический подбор	Сходство «задача ↔ опыт» через эмбеддинги не реализовано (необязательный пункт ТЗ); модели эмбеддингов в RouterAI доступны, таблица `embeddings` в схеме есть	Добавить компоненту с малым весом и кешем эмбеддингов
Регулярные задачи	Выдаются лениво при входе кандидата, без расписания; задачи по эталону проще слить, поэтому они влияют только на свежесть и доводы, а не на грейд	Cron-выдача и напоминания; параметризованные задачи с исполнением кода (см. `challenges.md`)

Область	Ограничение	Что с этим делать
Доступ к стенду	Домен `*.vercel.app` из части российских сетей открывается только через VPN (согласовано с организаторами)	Российский хостинг или собственный домен; локальный запуск — `docker compose up`

План развития

1. **Пилот с реальными кандидатами ФСП**: сбор ответов, калибровка, настройка весов индекса доверия и скоринга по откликам работодателей (какие приглашения принимают).
2. **Банк заданий**: по 60+ шаблонов на специализацию; Mobile, DevOps, аналитики; совместная разработка шаблонов с экспертами ФСП и ревью.
3. **Серверная песочница** для задач с ограничениями ресурсов и для повторной проверки кода модератором.
4. **Реальная интеграция с ФСП**: OIDC «ФСП ID», REST-реестр, вебхуки результатов.
5. **Подбор**: эмбеддинги «описание задачи ↔ опыт», обучение весов на исходах приглашений (learning-to-rank) с сохранением объяснимости через доводы.
6. **Работодатели**: верификация по ИНН, командные аккаунты, шаблоны приглашений, аналитика воронки.

13. Сценарий демонстрации

Подготовка:

- `npm run db:seed -- --reset` — свежие данные;
- два окна браузера: обычное и инкогнито;
- пароль всех демо-аккаунтов — `Demo12345`.

Время	Экран	Что говорим
0:00	Главная `/`	«Обычные джоб-сайты — это резюме и сотни откликов. Мы переворачиваем схему, как LHH: кандидат подтверждает уровень тестом, а работодатель сам находит его и приглашает — с зарплатой и с согласия кандидата на контакты».
0:30	Вход `candidate@demo.ru` → кабинет	«Анна заполнила профиль, но категории нет — значит, работодатели её не видят. Категория даётся только по тесту, не по резюме».
0:50	«Профиль» → «Загрузить PDF» (любое резюме)	«Резюме разбирает LLM через RouterAI; если ключа нет, работают правила. Контакты сразу уходят в защищённое хранилище, навыки помечаются „заявлено“».
1:20	«Тесты» → Backend, грейд выбираем сами → правила → «Начать»	«Грейд кандидат выбирает сам. Тест адаптивный, 10–18 заданий, час, задания приходят с сервера по одному. Вариант уникален: числа и данные сгенерированы для этой попытки».
1:50	Кодовое задание: Python → «Запустить на примерах»	«Код выполняется прямо в браузере — Pyodide в Web Worker с прерыванием по таймауту. Это лишь подсказка: при отправке сервер выдаёт скрытые входы, сгенерированные в этот момент, и сравнивает выходы со своим эталоном. Подделать результат можно, только решив задачу».
2:20	Попробовать скопировать условие; выйти из полноэкранного режима	«Копирование запрещено, выход из полноэкранного режима закрывает задание — всё это пишется в журнал. В условии спрятаны ловушки: персональный маркер для ИИ-ассистента и число-двойник, которое на экране одно, а в тексте страницы другое».
2:45	«Завершить досрочно» или показать готовую попытку `strong@demo.ru`	«Результат — балл из 30 на эталонном бланке грейда. Не сдал — сразу можно грейд ниже; сдал уверенно, от 25 из 30, — сразу можно выше; тот же грейд — через 14 дней».
3:05	Инкогнито: `employer@demo.ru` → «Новая потребность» → «Вставить текст вакансии»	«Работодатель вставляет текст вакансии — LLM раскладывает его в поля по схеме, работодатель проверяет. Без вилки зарплаты сохранить нельзя».
3:35	Подборка	«В подборке только кандидаты с категорией нужного грейда, соседний грейд — отдельно. У каждого match и доводы „за“ и „против“, собранные из компонент скоринга: что подтверждено тестом, что только заявлено, ФСП, зарплата. Контактов нет, имя скрыто».
4:05	Включить «Только с ФСП», порог теста, затем «Сбросить»	«Уточнение работает поверх сохранённой подборки — исходный список не теряется». «Доводы человеческим языком» — LLM перефразирует без новых фактов, это проверяется.
4:30	«Пригласить» Максима С. → отправить без вилки → ошибка → заполнить вилку	«Вилка обязательна на уровне API — без неё 422».

Время	Экран	Что говорим
4:55	Окно `strong@demo.ru` → «Приглашения» → открыть → «Принять»	«Кандидат видит компанию, задачу и зарплату. Статус стал „просмотрено“, теперь „принято“ — история видна обеим сторонам».
5:20	Окно работодателя → карточка приглашения	«Только теперь работодатель видит контакты, а в журнале доступа кандидата это записано».
5:30	`strong@demo.ru` → «Задачи»; работодатель → «Задачи» → оценить ответ	«Раз в 14 дней система сама выдаёт кандидату короткую задачу от работодателя его категории — наименее показанную, чтобы задачи не расходились. Ответ по эталону проверяется сразу, „подход“ оценивает работодатель. Хороший результат — „свежий сигнал“ в подборке; грейд меняет только тест».
5:40	`admin@demo.ru` → «Модерация» → попытка с low → «Похожие решения»	«Подозрительные попытки не банятся автоматически: модератор видит журнал, сработавшие ловушки, похожие решения других кандидатов и решает сам. Детектор помечает задания, если по ним резко выросла доля верных ответов — признак слива».
6:10	`docs/validation.md` (графики)	«Всё проверено на синтетике со скрытой истиной: P@5 подбора 0.58 против 0.24 у поиска по ключевым словам; тест-ретест 0.81; при сливе 20% шаблонов детектор нашёл все слитые. Ловушки поймали 100% копирующих в ИИ без ложных срабатываний на честных».
6:40	`/api/docs`	«API описан OpenAPI из тех же схем, что валидируют запросы. Поднимается одной командой `docker compose up`».

Если жюри хочет пройти тест само: «Тесты» → блок «Экспресс-проверка для демо» → 6 коротких заданий, ~5 минут, категория присваивается с пометкой. После этого кандидат сразу виден в подборке работодателя.

Запасной план, если нет сети к RouterAI или jsDelivr: LLM-функции работают на правилах. Для Python-задания переключиться на JavaScript (нативный, без CDN).